Save Point

Desktop Packaging & Release Builds

QuillFin · Prepared for Logan Young, Save Point Limited · 6 November 2026

`.github/workflows/build.yml` builds a Windows installer and a Linux AppImage automatically whenever a GitHub release is created, and attaches both to it, the same pattern used by the `Quiz-app-Ai` (Python) and `kioskSigningSystem` projects' own `build.yml` workflows.

Why this isn't quite the same as PyInstaller `--onefile`

Those other projects are Python, where PyInstaller genuinely bundles the interpreter and every dependency into one literal binary file. Flutter doesn't have an equivalent for desktop: `flutter build windows` / `flutter build linux` always produce an executable *plus* separate native library files (the Flutter engine, plugin libraries, ICU data) that all have to ship together in the same folder.

The practical fix, and what this workflow does, is package that folder so the *download experience* is still "one file":

- **Windows**: Inno Setup compiles the whole release folder into a single installer `.exe` (`windows/installer.iss`). Double-click it, it installs the app and adds Start Menu/desktop shortcuts, same as most Windows software people already know how to install.
- **Linux**: AppImage packages the whole release folder into one self-contained, directly-executable file (`linux/appimage/`). No install step at all: download it, `chmod +x`, run it.

Triggering a build

- **Real release**: create a GitHub release on the repo. Both jobs run and attach their output to that release automatically.
- **Test run without a release**: use the "Run workflow" button on the `Build` workflow in the Actions tab (this repo's `workflow_dispatch` trigger exists specifically for this). The built files are attached to that run as downloadable artifacts instead of a release, so you can verify a change to the workflow or the installer scripts without cutting a real version first.

Known limitation: this hasn't been run yet

Neither the Windows nor the Linux job could be verified end-to-end while writing this. Windows desktop builds can only be compiled *on* Windows (Flutter doesn't support cross-compiling for it), and there was no Windows machine available to build one. The Linux desktop toolchain (`clang` / `cmake` / `ninja` / `libgtk-3-dev`) also needs root packages that weren't available in the

environment this was written in, though GitHub's own `ubuntu-22.04` runners have full root, so that install step should work fine there even though it couldn't be proven locally.

`flutter analyze` and `flutter test` were run after adding Windows platform support and pass; what's specifically unverified is the two platform-native build steps themselves and the installer/AppImage packaging around them. **Trigger a manual `workflow_dispatch` run before trusting a real release build.** If either job fails, the Actions tab's log is the place to start, and both `installer.iss` and the AppImage steps have comments explaining what each piece is for.

App icon

`linux/appimage/icon.png` is a placeholder (a flat 256x256 square in the app's purple accent color), generated programmatically since no real app icon exists yet. Replace it with a real one when the visual design gets settled (see `gui_theming.md` / `future_work.md` on the unverified savepoint.cc brand assets). The Windows installer doesn't currently set a custom icon at all; Inno Setup's default installer icon is used.

Distribution vs. app store submission

This workflow gets you a downloadable installer/AppImage attached to a GitHub release, which is genuinely how many small apps are distributed. It is not the same as publishing to the Microsoft Store, a Linux package repository, or (for the mobile builds this app is primarily aimed at) the Apple App Store or Google Play. Those all need their own developer accounts and submission processes; see `future_work.md`.